

SET-7 — MASTER PRACTICAL GUIDE

Apartment Management System

Windows PC • Eclipse • Git/GitHub • Maven • Tomcat • Docker

Repository	https://github.com/sarasrija/ApartmentManagementSystem.git
Coverage	44 individual task entries, each followed immediately by an answer
Exam strategy	Command-first, GUI-supported, with older-version fallbacks and examiner-proof checks

Use this as the exam-day walkthrough: open Git Bash/Command Prompt, Eclipse, Docker Desktop, and a browser side by side. For every task, perform the command, verify the result, and only then move on.

HOW TO USE THIS MASTER PDF

This guide is written for a Windows PC and is designed for practical-exam demonstration. Commands are the most version-tolerant method; GUI routes are included where Eclipse/Docker Desktop can be used. Exact menu labels can differ slightly between old and new releases, so use the command equivalent if a GUI label is missing.

Important source limitation: several exam questions refer to a “provided pom.xml”, but the uploaded question papers contain the questions, not the actual POM file. For those questions, this guide deliberately shows how to derive the exact value from the supplied POM instead of inventing coordinates, WAR names, or context paths.

1. Windows tool setup

Tool	What to verify	Windows command / GUI
Java JDK	Use a JDK, not only a JRE.	<code>java -version</code> <code>javac -version</code>
Maven	Maven available to the terminal, or Maven Wrapper exists.	<code>mvn -version</code> <code>mvnw.cmd -version</code>
Git	Git Bash/Command Prompt can run Git.	<code>git --version</code>
Eclipse	Maven integration (m2e) and Java support.	Window → Preferences → Java
Tomcat	Choose a Tomcat version compatible with the project.	Servers view → New → Apache Tomcat
Docker	Docker daemon/Desktop is running.	<code>docker version</code> Docker Desktop GUI

If `mvn` is not recognized

Either install/configure Maven and add its **bin** directory to Windows PATH, or check whether the repository contains **mvnw.cmd**. The wrapper is often the safest option for older student projects because it uses the project's expected Maven distribution.

```
where mvn
mvn -version
mvnw.cmd -version
```

Windows GUI: Control Panel / Settings → System → Advanced system settings → Environment Variables → create MAVEN_HOME=C:\apache-maven-3.x.x → edit Path → add %MAVEN_HOME%\bin → reopen terminal.

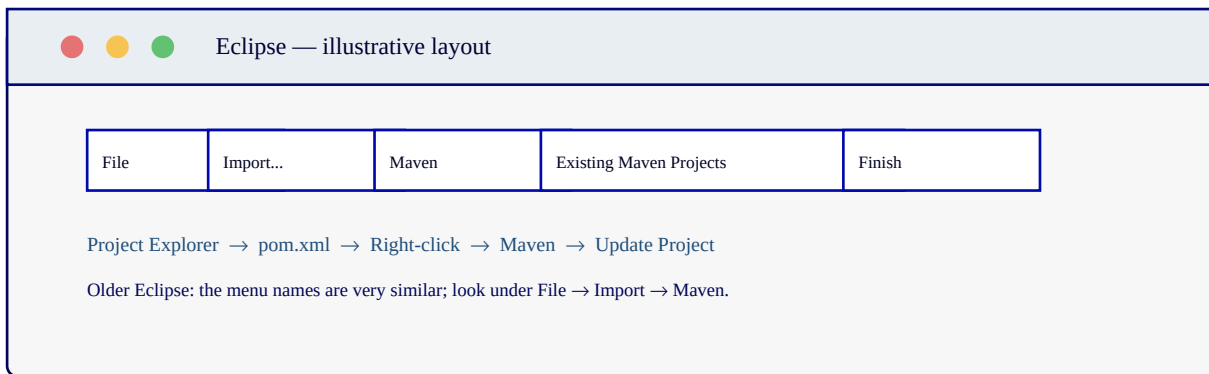
Java compatibility rule

Always run **mvn -version**, not only **java -version**, because Maven reports the Java home it actually uses. If an old project expects Java 8/11 and a newer JDK is installed, align the JDK used by Maven and Eclipse with the project's compiler settings. Do not blindly change the POM until you know the required source/target/release level.

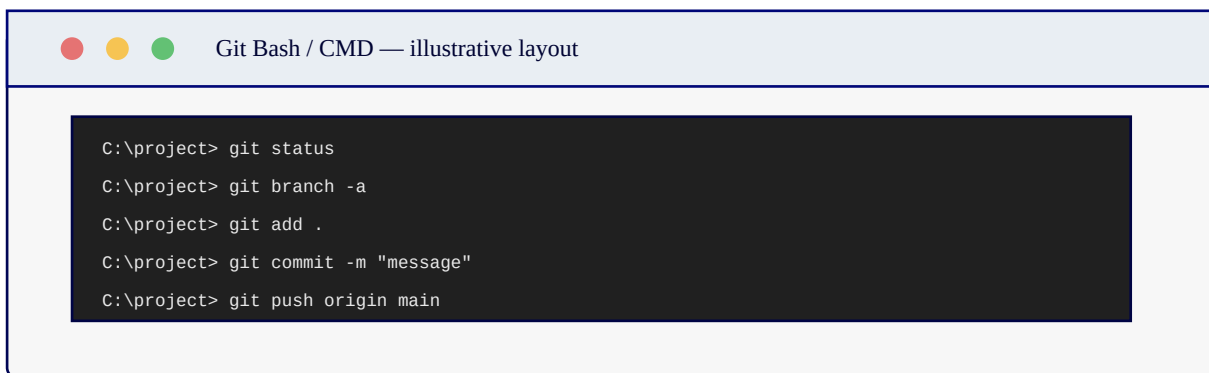
Tomcat compatibility warning

Older JSP/Servlet projects commonly use the **javax.servlet** API. Tomcat 10+ moved to the **jakarta.servlet** namespace. If the project imports `javax.servlet.*`, Tomcat 9 (or another compatible javax-based server) is usually the safer exam choice unless the code has already been migrated to Jakarta.

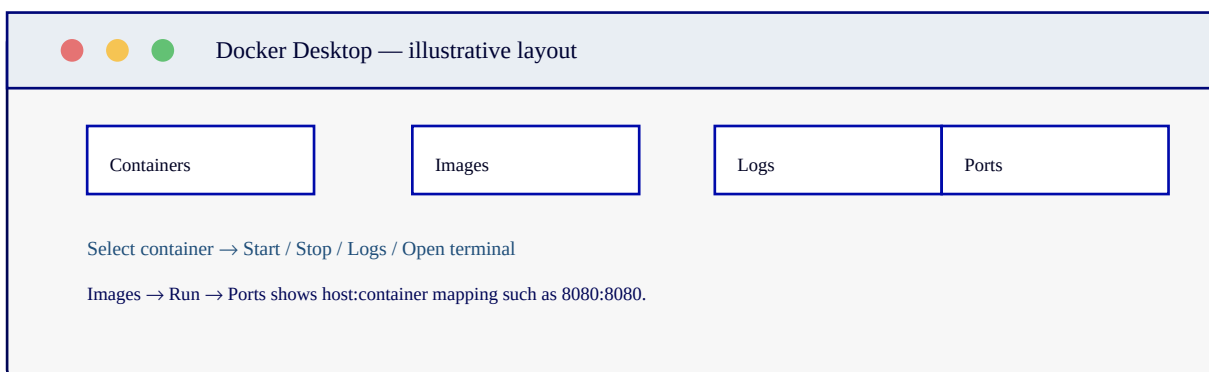
2. GUI routes you can demonstrate to the examiner



Eclipse Maven import: **File** → **Import** → **Maven** → **Existing Maven Projects** → Browse to the cloned folder → select pom.xml → Finish. Then right-click the project → Maven → Update Project. For older Eclipse releases, the wording is almost identical.



Git tasks are best demonstrated in Git Bash because every advanced command in these papers is available there. GitHub Desktop/Eclipse EGit can perform many common operations, but rebase, reflog, patch, and detailed recovery are much easier to prove from Git Bash.



Docker Desktop is useful for showing Images, Containers, Logs, and port mappings. If the lab machine has an older Docker Desktop layout, the same objects may appear under older tabs/menu names. The CLI remains the most reliable fallback.

3. Universal exam command sheet

Area	Commands
Git basics	<code>git clone URL</code> <code>cd project</code> <code>git status</code> <code>git remote -v</code> <code>git branch -a</code>
Branching	<code>git switch -c feature/name</code> # older Git: <code>git checkout -b feature/name</code>
Inspect changes	<code>git diff</code> <code>git diff --cached</code> <code>git status</code>
Commit	<code>git add file</code> <code>git commit -m "message"</code> <code>git log -1 --oneline</code>
Remote sync	<code>git fetch origin</code> <code>git merge origin/main</code> <code>git pull origin main</code>
Rebase	<code>git fetch origin</code> <code>git rebase origin/main</code> <code>git rebase --abort</code>
Undo safely	<code>git revert <hash></code> <code>git reset --soft HEAD~1</code> <code>git reflog</code>
Stash	<code>git stash push -m "work"</code> <code>git stash list</code> <code>git stash pop</code>
Compare/history	<code>git diff main feature -- file</code> <code>git log --oneline --graph --decorate --all</code>
Maven	<code>mvn -version</code> <code>mvn clean package</code> <code>mvn dependency:tree</code> <code>mvn dependency:analyze</code> <code>mvn clean package -X</code>
Tests	<code>mvn test</code> <code>mvn -Dtest=ClassName test</code> <code>mvn clean package -DskipTests</code>
Docker	<code>docker build -t app:v1 .</code> <code>docker run -d --name app -p 8080:8080 app:v1</code> <code>docker ps</code> <code>docker logs app</code>
Docker Hub	<code>docker login</code> <code>docker tag app:v1 user/app:v1</code> <code>docker push user/app:v1</code> <code>docker logout</code>

4. Universal troubleshooting decision tree

Maven failure: read the first meaningful [ERROR] → identify the phase/goal → inspect the POM coordinates/configuration → run `dependency:tree/effective-pom` → check JDK with `mvn -version` → clean → rebuild with `-X`.

Browser failure: prove Maven created the artifact → prove the server started → prove the port is listening → prove the context path → read server logs → then inspect JSP/application code.

Docker failure: prove Docker daemon → prove Dockerfile/context → prove image exists → prove container exists → inspect `docker ps -a` → inspect docker logs → inspect port mapping.

5. COMPLETE TASK-BY-TASK QUESTION → ANSWER GUIDE

The task wording below is kept close to the uploaded question paper while being shortened where a long scenario is repeated. Every numbered/sub-numbered practical requirement is represented. Where the source asks for an exact value that depends on an unseen POM, the answer explicitly tells you how to derive it.

Git1 — TASK / QUESTION

Set up the codebase on the local machine.

ANSWER / PROCEDURE

1) Install Git for Windows. Open Git Bash. 2) Move to the folder where you want the project. 3) Clone the repository. 4) Enter the folder and verify the remote. 5) In Eclipse use File → Import → Maven → Existing Maven Projects (older Eclipse: File → Import → Maven → Existing Maven Projects), select the folder containing pom.xml, Finish, then Project → Clean if necessary.

Commands / configuration

```
git clone https://github.com/sarasrija/ApartmentManagementSystem.git
cd ApartmentManagementSystem
git status
git remote -v
```

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Git2 — TASK / QUESTION

Verify the remote server connections used for fetching and pushing.

ANSWER / PROCEDURE

Use `git remote -v` to show fetch/push URLs. Use `git remote show origin` for a more detailed remote/branch view.

Commands / configuration

```
git remote -v
git remote show origin
```

GUI route: Eclipse → Window → Show View → Servers → add/start Tomcat; then open the URL in a browser and check the Servers/Console view for startup errors.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Git3 — TASK / QUESTION

Inspect staged, unstaged, and untracked changes after modifying src/main/webapp/index.jsp and creating src/main/webapp/complaints.jsp.

ANSWER / PROCEDURE

Use `git status` first. It separates modified-but-unstaged files, staged files, and untracked files. For exact line-level edits use `git diff`; for staged edits use `git diff --cached`.

Commands / configuration

```
git status
git diff
git diff --cached
```

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Git4 — TASK / QUESTION

Create and switch to feature/tenant-registration in one operation.

ANSWER / PROCEDURE

Create and switch in one operation. This is the modern command; on older Git use the checkout equivalent.

Commands / configuration

```
git switch -c feature/tenant-registration
git checkout -b feature/tenant-registration
```

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Git5 — TASK / QUESTION

List all local and remote branches and confirm the active branch.

ANSWER / PROCEDURE

`git branch` lists local branches, `git branch -r` remote-tracking branches, and `git branch -a` both. The current branch is marked with `\*`. Modern Git also has `git branch --show-current`.

Commands / configuration

```
git branch -a
git branch --show-current
git rev-parse --abbrev-ref HEAD
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Git6 — TASK / QUESTION

Commit src/main/java/com/apartment/servlet/TenantServlet.java with the requested message.

ANSWER / PROCEDURE

Stage only the requested file, then create the commit. Verify the commit immediately with `git log` or `git show`.

Commands / configuration

```
git add "src/main/java/com/apartment/servlet/TenantServlet.java"
git commit -m "Add tenant registration servlet"
git log -1 --oneline
git show --stat HEAD
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Git7 — TASK / QUESTION

Add src/main/webapp/WEB-INF/web.xml to the previous commit without changing its message.

ANSWER / PROCEDURE

Stage the omitted file and amend the previous commit. `--no-edit` preserves the original commit message.

Commands / configuration

```
git add "src/main/webapp/WEB-INF/web.xml"
git commit --amend --no-edit
git show --stat HEAD
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Git8 — TASK / QUESTION

Fetch a teammate's updated pom.xml from GitHub without changing local working files.

ANSWER / PROCEDURE

Fetch downloads new remote objects and references but does not merge them into your current working tree. This is the safe answer when the question says 'retrieve without modifying local files'.

Commands / configuration

```
git fetch origin
git status
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Git9 — TASK / QUESTION

Merge the latest origin/main into the currently active branch.

ANSWER / PROCEDURE

After fetching, merge the remote main branch into the current branch. If the question explicitly wants a simple update, `git pull origin main` performs fetch + merge.

Commands / configuration

```
git fetch origin
git merge origin/main
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Git10 — TASK / QUESTION

Rebase the feature branch so it starts from the tip of the updated main branch.

ANSWER / PROCEDURE

Fetch first, then rebase your feature branch onto the updated main branch. This rewrites the feature branch commit IDs, so do it before sharing the branch or be prepared to use `--force-with-lease` when pushing.

Commands / configuration

```
git fetch origin
git switch feature/your-branch
git rebase origin/main
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Git11 — TASK / QUESTION

Abort a broken rebase and return to the pre-rebase state.

ANSWER / PROCEDURE

If a rebase is in progress and you want to abandon it completely, abort it. The branch is returned to the state it had before the rebase started.

Commands / configuration

```
git rebase --abort
```

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Git12 — TASK / QUESTION

Undo the faulty commit in src/test/java/com/apartment/TenantTest.java while preserving Git history.

ANSWER / PROCEDURE

Use `git revert` when the commit has already been recorded/shared and you want to preserve history. Git creates a new commit that reverses the old commit.

Commands / configuration

```
git log --oneline
```

```
git revert <commit-hash>
git log --oneline -2
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Git13 — TASK / QUESTION

Undo the last commit while keeping all modified changes staged.

ANSWER / PROCEDURE

`--soft` moves HEAD back but leaves the changes staged in the index. It is the correct choice when the question explicitly says to undo the commit while keeping changes staged.

Commands / configuration

```
git reset --soft HEAD~1
git status
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Git14 — TASK / QUESTION

Stop tracking src/main/resources/db-config.env but keep the file on the local drive.

ANSWER / PROCEDURE

Remove the file from Git tracking but leave the physical file on disk. Then add the pattern to `.gitignore` so it is not accidentally committed again.

Commands / configuration

```
git rm --cached "src/main/resources/db-config.env"
echo db-config.env >> .gitignore
git add .gitignore
git commit -m "Stop tracking local configuration"
```

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Git15 — TASK / QUESTION

Stash unfinished changes in complaints.jsp so you can switch branches without creating a commit.

ANSWER / PROCEDURE

Stash saves uncommitted work without creating a commit. Give the stash a name so it is easy to recognize.

Commands / configuration

```
git stash push -m "unfinished work"
git switch <emergency-branch>
git stash list
git stash pop
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Git16 — TASK / QUESTION

List all stashes and restore the saved unfinished work.

ANSWER / PROCEDURE

Stash saves uncommitted work without creating a commit. Give the stash a name so it is easy to recognize.

Commands / configuration

```
git stash push -m "unfinished work"
git switch <emergency-branch>
git stash list
git stash pop
```

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Git17 — TASK / QUESTION

Compare feature/tenant-registration and main line-by-line for src/main/webapp/index.jsp.

ANSWER / PROCEDURE

Compare the two branch tips and restrict the diff to the requested file. If the examiner asks for a true merge-base comparison, use the three-dot form.

Commands / configuration

```
git diff main feature/tenant-registration -- "src/main/webapp/index.jsp"
git diff main...feature/tenant-registration -- "src/main/webapp/index.jsp"
```

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Git18 — TASK / QUESTION

Show a compact one-line visual branch/merge timeline.

ANSWER / PROCEDURE

The compact one-line visual history is produced by the graph/log command. `--all` includes all local and remote-tracking refs.

Commands / configuration

```
git log --oneline --graph --decorate --all
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Git19 — TASK / QUESTION

Return to main and merge the completed feature/tenant-registration feature.

ANSWER / PROCEDURE

Switch to main, make sure it is current, merge the feature branch, resolve conflicts if Git stops, then commit the merge if required.

Commands / configuration

```
git switch main
git pull origin main
git merge feature/tenant-registration
git status
git push origin main
```

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Git20 — TASK / QUESTION

Push updated main to GitHub and verify local and remote point to the same commit.

ANSWER / PROCEDURE

Push the branch and then compare the local commit ID with the remote-tracking branch after fetching. Equal hashes mean both point to the same commit.

Commands / configuration

```
git push -u origin main
git fetch origin
git rev-parse HEAD
```

```
git rev-parse origin/main
git status
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run ``git status`` and/or ``git log -1 --oneline``; show the relevant branch/remote/diff output. Do not only say the command — show its result.

M1 — TASK / QUESTION

``mvn validate`` fails while parsing pom.xml. Identify the XML schema/nesting problem and write the corrected dependency configuration.

ANSWER / PROCEDURE

Run Maven with debug logging and preserve the first meaningful error. Do not start by changing random POM values. Check the first `[ERROR]`, the goal/phase named, and the ``Caused by:`` chain.

Commands / configuration

```
mvn clean package -X
mvn clean package -e
mvn -version
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

M2 — TASK / QUESTION

``mvn tomcat7:run`` fails because Maven cannot execute the plugin. Explain the cause and give the corrected plugin block.

ANSWER / PROCEDURE

For a plugin execution failure, inspect ``<build><plugins>`` and the plugin's ``<groupId>``, ``<artifactId>``, ``<version>``, and ``<configuration>``. ``pluginManagement`` only supplies defaults; it does not activate a plugin by itself. If the command uses ``tomcat7:run``, the plugin normally needs the Apache Tomcat Maven Plugin coordinates.

Commands / configuration

```
mvn help:effective-pom
mvn tomcat7:run
```

GUI route: Eclipse → Window → Show View → Servers → add/start Tomcat; then open the URL in a browser and check the Servers/Console view for startup errors.

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

M3 — TASK / QUESTION

``mvn compile`` cannot resolve the MySQL database driver. Explain the artifact-resolution issue and correct the dependency coordinates from the provided POM.

ANSWER / PROCEDURE

Use the dependency tree to see which paths bring different versions. Maven normally selects the nearest definition in the dependency graph; a `dependencyManagement` entry or explicit direct dependency can be used to pin the intended version. Exclusions remove an unwanted transitive path.

Commands / configuration

```
mvn dependency:tree -Dverbose
mvn help:effective-pom
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

POM-dependent note: the uploaded question paper does not include the actual POM contents. In the exam, read the provided POM and substitute its real groupId/artifactId/version/finalName/plugin path. Do not invent a value merely to make the answer look exact.

M4 — TASK / QUESTION

``mvn test-compile`` fails because a dependency declaration is incomplete. Identify the missing element and correct the dependency block.

ANSWER / PROCEDURE

Run Maven with debug logging and preserve the first meaningful error. Do not start by changing random POM values. Check the first `[ERROR]`, the goal/phase named, and the ``Caused by:`` chain.

Commands / configuration

```
mvn clean package -X
mvn clean package -e
mvn -version
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

M5 — TASK / QUESTION

After ``mvn clean package``, determine the exact WAR filename and the rule used to construct it.

ANSWER / PROCEDURE

By default, a Maven WAR artifact is named ``<artifactId>-<version>.war`` in ``target/``. If ``<build><finalName>`` is present, that value replaces the default base name.

Commands / configuration

```
dir target
mvn help:evaluate -Dexpression=project.build.finalName -q -DforceStdout
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

M6 — TASK / QUESTION

Determine the normal local-server context-path URL from the embedded web-server plugin configuration.

ANSWER / PROCEDURE

For an embedded Tomcat plugin, inspect the plugin configuration for a ``path/`` `contextPath`` value. If none is configured, the context commonly follows the artifact/project name. Do not guess an exact URL when the POM is not shown; demonstrate how to read it from the provided POM.

Commands / configuration

```
mvn help:effective-pom
```

GUI route: Eclipse → Window → Show View → Servers → add/start Tomcat; then open the URL in a browser and check the Servers/Console view for startup errors.

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

M7 — TASK / QUESTION

Explain pluginManagement versus plugins and explain the -SNAPSHOT suffix.

ANSWER / PROCEDURE

``pluginManagement`` defines defaults/configuration that child projects can inherit; it does not cause a plugin goal to run merely because it is listed there. ``<plugins>`` declares the plugin as active in the current project.

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

M8 — TASK / QUESTION

Give the Maven JDK inspection command and the single clean/package/debug command.

ANSWER / PROCEDURE

First compare the JDK Maven actually uses with the project compiler level. ``mvn -version`` is better than only ``java -version`` because it reports the Java home used by Maven. Then align Eclipse's installed JRE/JDK and the POM compiler settings.

Commands / configuration

```
java -version
javac -version
mvn -version
where java
where mvn
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

M9 — TASK / QUESTION

Install vendor library apartment-analytics.jar into the local Maven repository using the coordinates specified in the POM.

ANSWER / PROCEDURE

If a vendor JAR is not in a remote Maven repository, install it into your local Maven repository using the exact groupId/artifactId/version/type coordinates from the POM. Replace the placeholders below with the coordinates shown in the exam's POM; do not invent them.

Commands / configuration

```
mvn install:install-file -Dfile="target/apartment-analytics.jar" -DgroupId=<groupId> -DartifactId=<artifactId>
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

POM-dependent note: the uploaded question paper does not include the actual POM contents. In the exam, read the provided POM and substitute its real groupId/artifactId/version/finalName/plugin path. Do not invent a value merely to make the answer look exact.

M10 — TASK / QUESTION

Show the complete direct/transitive dependency tree.

ANSWER / PROCEDURE

The dependency tree shows direct and transitive dependencies and is the quickest way to investigate version conflicts.

Commands / configuration

```
mvn dependency:tree
mvn dependency:tree -Dverbose
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

M11 — TASK / QUESTION

Give the locations of compiled test classes and JUnit reports; run only MaintenanceBillingTest; and state the flag that packages despite test failures.

ANSWER / PROCEDURE

Maven's conventional locations are `target/test-classes` for compiled test classes and `target/surefire-reports` for JUnit/Surefire reports.

Commands / configuration

```
dir target\test-classes
dir target\surefire-reports
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

M12 — TASK / QUESTION

Analyze dependency usage and explain why javax.servlet-api is normally provided by Tomcat.

ANSWER / PROCEDURE

Use the Dependency Plugin's analyze goal to report used/unused declared dependencies. Treat the result as a diagnostic: reflection, generated code, and runtime/container-provided APIs can make a dependency look unused even when it is needed.

Commands / configuration

```
mvn dependency:analyze
```

GUI route: Eclipse → Window → Show View → Servers → add/start Tomcat; then open the URL in a browser and check the Servers/Console view for startup errors.

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

D1 — TASK / QUESTION

Clone the repository, enter the project directory, and confirm essential configuration files exist.

ANSWER / PROCEDURE

1) Install Git for Windows. Open Git Bash. 2) Move to the folder where you want the project. 3) Clone the repository. 4) Enter the folder and verify the remote. 5) In Eclipse use File → Import → Maven → Existing Maven Projects (older Eclipse: File → Import → Maven → Existing Maven Projects), select the folder containing pom.xml, Finish, then Project → Clean if necessary.

Commands / configuration

```
git clone https://github.com/sarasrija/ApartmentManagementSystem.git
cd ApartmentManagementSystem
git status
git remote -v
```

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

D2 — TASK / QUESTION

Write the Dockerfile using openjdk:17-jdk-slim, /app, and target/apartment-management.jar; define the startup instruction.

ANSWER / PROCEDURE

If the question explicitly asks for `openjdk:17-jdk-slim` and a JAR, use a simple runtime image. The JAR must exist before `docker build`. For older Docker engines the syntax below is broadly compatible.

Commands / configuration

```
FROM openjdk:17-jdk-slim
WORKDIR /app
COPY target/apartment-management.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

D3 — TASK / QUESTION

Build the custom image apartmentapp-image from the current directory.

ANSWER / PROCEDURE

Run the build from the directory containing the Dockerfile and required build artifacts. Then verify the image ID, tag, and size.

Commands / configuration

```
docker build -t apartmentapp-image:v1 .
docker images apartmentapp-image
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

D4 — TASK / QUESTION

Run apartment-app-container in the background with host 8080 mapped to container 8080.

ANSWER / PROCEDURE

Start detached, map host 8080 to container 8080, and name the container so it is easy to demonstrate.

Commands / configuration

```
docker run -d --name apartment-app-container -p 8080:8080 apartmentapp-image:v1
docker ps
docker port apartment-app-container
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

D5 — TASK / QUESTION

Display all running containers.

ANSWER / PROCEDURE

`docker ps` shows running containers. `docker ps -a` includes stopped/exited containers.

Commands / configuration

```
docker ps
docker ps -a
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

D6 — TASK / QUESTION

Display all containers, including stopped ones.

ANSWER / PROCEDURE

`docker ps` shows running containers. `docker ps -a` includes stopped/exited containers.

Commands / configuration

```
docker ps
docker ps -a
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

D7 — TASK / QUESTION

Enter apartment-app-container interactively using a shell.

ANSWER / PROCEDURE

Enter the running Linux container with an interactive shell. If `/bin/bash` is absent in a minimal image, use `/bin/sh`.

Commands / configuration

```
docker exec -it apartment-app-container /bin/bash
docker exec -it apartment-app-container /bin/sh
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

D8 — TASK / QUESTION

Stop apartment-app-container and restart it.

ANSWER / PROCEDURE

Stop and restart the same container. `start` reuses the existing container; it does not create a new one.

Commands / configuration

```
docker stop apartment-app-container
docker start apartment-app-container
docker ps
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

D9 — TASK / QUESTION

Snapshot container ID 0e993d2009a1 into your Docker Hub namespace as apartmentapp:v1.

ANSWER / PROCEDURE

`docker commit` snapshots the current container filesystem into a new image. In an exam, state that a Dockerfile is normally preferred for reproducibility; commit is used here because the question explicitly asks for a snapshot.

Commands / configuration

```
docker commit 0e993d2009a1 <dockerhub-username>/apartmentapp:v1
docker images
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

D10 — TASK / QUESTION

Authenticate the terminal session with Docker Hub.

ANSWER / PROCEDURE

Authenticate, tag the local image with your Docker Hub namespace, push it, then verify on Docker Hub or with `docker pull` from another machine.

Commands / configuration

```
docker login
docker tag apartmentapp-image:v1 <dockerhub-username>/apartmentapp:v1
docker push <dockerhub-username>/apartmentapp:v1
docker pull <dockerhub-username>/apartmentapp:v1
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

D11 — TASK / QUESTION

Push your Docker Hub image apartmentapp:v1 to the public repository.

ANSWER / PROCEDURE

Authenticate, tag the local image with your Docker Hub namespace, push it, then verify on Docker Hub or with `docker pull` from another machine.

Commands / configuration

```
docker login
docker tag apartmentapp-image:v1 <dockerhub-username>/apartmentapp:v1
docker push <dockerhub-username>/apartmentapp:v1
docker pull <dockerhub-username>/apartmentapp:v1
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

D12 — TASK / QUESTION

Log out and remove the stored Docker Hub credentials from the local session.

ANSWER / PROCEDURE

Use ``docker logout`` to remove the saved registry credentials for the default Docker Hub registry from the local Docker credential configuration.

Commands / configuration

```
docker logout
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show ``docker images`` or ``docker ps -a``, then show logs/ports when relevant. For browser tasks, actually open ``http://localhost:8080``.

6. EXAMINER DEMONSTRATION CHECKLIST

- Open the project folder and prove the repository/remote is correct.
- Open pom.xml and point to groupId, artifactId, version, packaging, dependencies and plugins when asked.
- Run mvn -version and, if required, java -version/javac -version.
- Build with mvn clean package (or the exact lifecycle command in the question).
- Show target/ and the generated WAR/JAR.
- In Eclipse, use Maven → Update Project when dependencies are stale.
- For Git, show status/diff before commit and log after commit.
- For rebase/merge questions, show the branch name and final clean status.
- For Docker, show docker images, docker ps -a, docker logs and port mapping as appropriate.
- Open http://localhost:8080 only after proving the server/container is running.
- For Docker Hub, show tag → login → push → repository verification.

7. OLD-VERSION FALLBACKS

If you see...	Use this fallback
Old Git without `git switch`	Use `git checkout -b branch` / `git checkout branch`.
Old Eclipse	Look under File → Import → Maven and Window → Show View → Servers; labels vary slightly.
Old Maven project	Check for `mvnw.cmd`; otherwise use a Maven version compatible with the project/JDK.
Old Tomcat project using javax.servlet	Prefer a javax-compatible Tomcat release (commonly Tomcat 9) unless the project is migrated to Jakarta.
Docker Desktop GUI differs	Use the CLI; the core objects remain images, containers, logs, and port mappings.
PowerShell command differs	Git Bash is the simplest universal shell for the Git commands in these papers.

Official references used for the practical workflow

Git for Windows: <https://git-scm.com/install/windows>

Eclipse documentation: <https://www.eclipse.org/documentation/>

Maven Compiler Plugin: <https://maven.apache.org/plugins/maven-compiler-plugin/>

Maven Dependency Plugin: <https://maven.apache.org/plugins/maven-dependency-plugin/>

Docker Desktop for Windows: <https://docs.docker.com/desktop/setup/install/windows-install/>